

	MINI PROJECT
	SEPARATION OF NOISES FROM A SONG USING MATLAB

AIM:

To develop a MATLAB program that separates noises from a single audio recording using bandpass filtering, and analyzes their frequency characteristics by plotting the magnitude and phase spectra of both the original and separated signals.

SOFTWARE REQUIRED:

MATLAB ONLINE

PROGRAM:

% Mini Project: Add Noise to Audio, Filter it, and Save Output

% Step 1: Load your audio file

[audio, Fs] = audioread('ponni_nadhi_ringtone.wav'); % Ensure 'audio.wav' exists in the same folder

audio = audio(:,1); % Use only one channel (mono)

N = length(audio);

t = (0:N-1)/Fs;

% Step 2: Generate white Gaussian noise

noise_level = 0.02; % Adjust for more/less noise

noise = noise_level * randn(size(audio));

% Step 3: Plot original audio and noise in time domain

figure;

subplot(2,1,1);

plot(t, audio);

title('Original Audio - Time Domain');

xlabel('Time (s)');

ylabel('Amplitude');

```
subplot(2,1,2);  
plot(t, noise);  
title('Generated Noise - Time Domain');  
xlabel('Time (s)');  
ylabel('Amplitude');  
  
% Step 4: Frequency domain (FFT of both signals)  
f = Fs*(0:(N/2))/N;  
audio_fft = fft(audio);  
noise_fft = fft(noise);  
  
figure;  
subplot(2,1,1);  
plot(f, abs(audio_fft(1:N/2+1)));  
title('Original Audio - Frequency Domain');  
xlabel('Frequency (Hz)');  
ylabel('|Amplitude|');  
  
subplot(2,1,2);  
plot(f, abs(noise_fft(1:N/2+1)));  
title('Noise - Frequency Domain');  
xlabel('Frequency (Hz)');  
ylabel('|Amplitude|');  
  
% Step 5: Add noise to audio signal  
noisy_signal = audio + noise;  
  
% Step 6: Plot noisy signal (time and frequency)  
noisy_fft = fft(noisy_signal);
```

```
figure;
subplot(2,1,1);
plot(t, noisy_signal);
title('Noisy Signal - Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');

subplot(2,1,2);
plot(f, abs(noisy_fft(1:N/2+1)));
title('Noisy Signal - Frequency Domain');
xlabel('Frequency (Hz)');
ylabel('|Amplitude|');

% Step 7: Filter the noisy signal using low-pass filter
cutoff = 3000; % Hz (adjust based on audio content)
[b, a] = butter(6, cutoff/(Fs/2), 'low');
filtered_signal = filtfilt(b, a, noisy_signal);

% Step 8: Plot filtered signal (time and frequency)
filtered_fft = fft(filtered_signal);
figure;
subplot(2,1,1);
plot(t, filtered_signal);
title('Filtered Signal - Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');

subplot(2,1,2);
plot(f, abs(filtered_fft(1:N/2+1)));
title('Filtered Signal - Frequency Domain');
```

```
xlabel('Frequency (Hz)');  
ylabel('|Amplitude|');
```

```
% Step 9: Listen to the signals (optional)
```

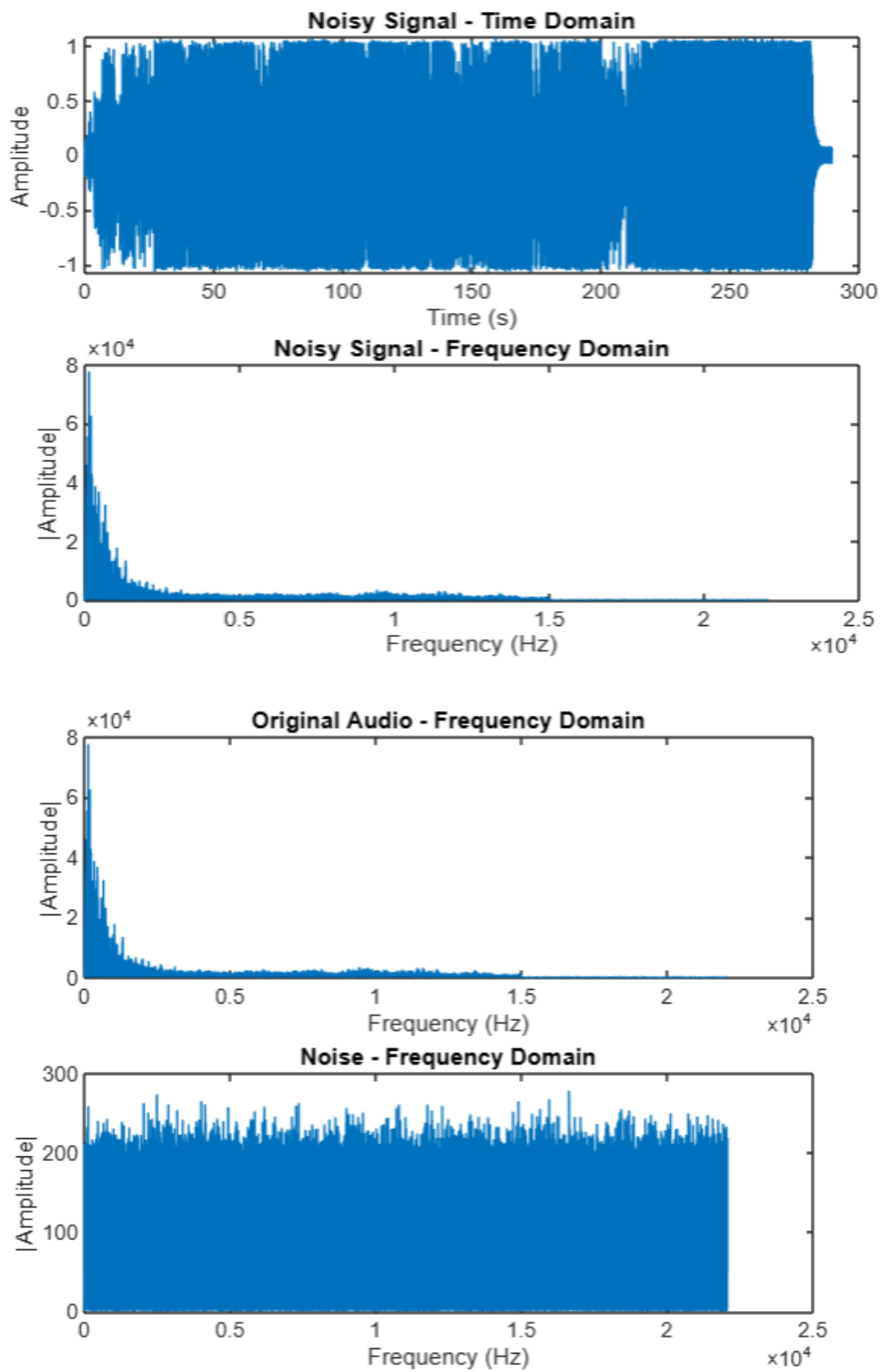
```
disp('Playing original audio...');  
sound(audio, Fs);  
pause(N/Fs + 1);
```

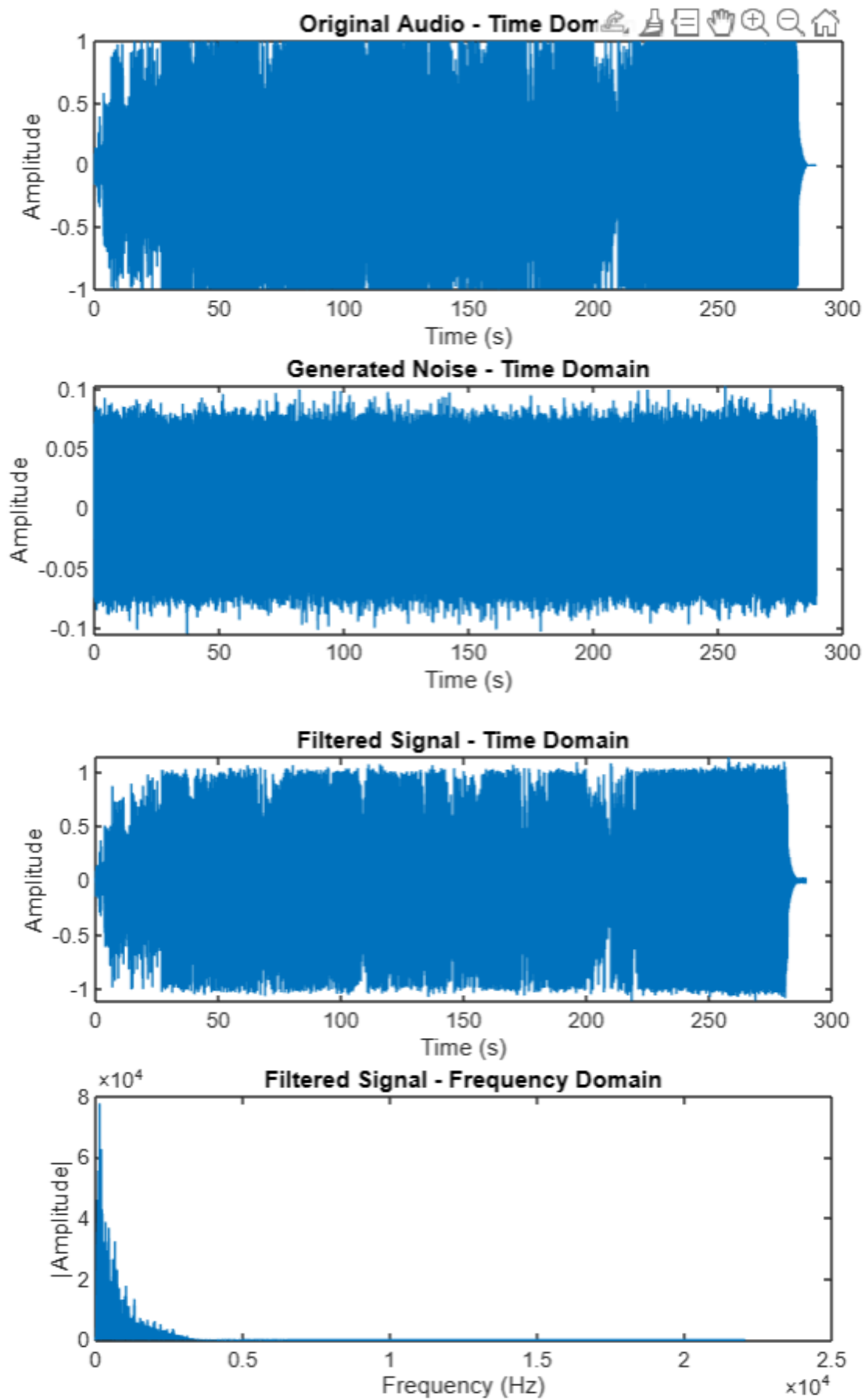
```
disp('Playing noisy audio...');  
sound(noisy_signal, Fs);  
pause(N/Fs + 1);
```

```
disp('Playing filtered audio...');  
sound(filtered_signal, Fs);  
pause(N/Fs + 1);
```

```
% Step 10: Save the filtered signal as .wav
```

```
filtered_signal = filtered_signal / max(abs(filtered_signal)); % Normalize  
audiowrite('filtered_output.wav', filtered_signal, Fs);  
disp('✅ Filtered audio saved as "filtered_output.wav".');
```

OUTPUT:



PROGRAM FOR REAL TIME SONG AND NOISE:

```
clc; clear; close all;
```

```
%% Step 1: Load Song and Noise
```

```
[song, Fs1] = audioread('song.wav');
```

```
[noise, Fs2] = audioread('noise_signal.wav');
```

```
% Convert stereo to mono
```

```
if size(song, 2) > 1
```

```
    song = mean(song, 2);
```

```
end
```

```
if size(noise, 2) > 1
```

```
    noise = mean(noise, 2);
```

```
end
```

```
% Resample noise to match song sample rate
```

```
if Fs1 ~= Fs2
```

```
    noise = resample(noise, Fs1, Fs2);
```

```
end
```

```
% Match lengths
```

```
min_len = min(length(song), length(noise));
```

```
song = song(1:min_len);
```

```
noise = noise(1:min_len);
```

```
%% Step 2: Create and Save Noisy Signal
```

```
noisy_signal = song + 0.4 * noise;
```

```
noisy_signal = noisy_signal / max(abs(noisy_signal));
```

```
audiowrite('noisy_song.wav', noisy_signal, Fs1);
```

```
disp('✅ Noisy song saved.');
```

```
%% Step 3: STFT-Domain Wiener Filtering
```

```
window = hamming(512);
```

```
nfft = 1024;
```

```
overlap = 256;
```

```
% STFT
```

```
[S_noisy, F, T] = stft(noisy_signal, Fs1, 'Window', window, 'OverlapLength', overlap, 'FFTLenght', nfft);
```

```
[S_noise, ~, ~] = stft(noise, Fs1, 'Window', window, 'OverlapLength', overlap, 'FFTLenght', nfft);
```

```
% Power Spectral Density
```

```
P_noisy = abs(S_noisy).^2;
```

```
P_noise = abs(S_noise).^2;
```

```
% Estimate clean signal via Wiener gain
```

```
G = max((P_noisy - P_noise), 0) ./ (P_noisy + eps); % Avoid division by zero
```

```
S_clean = G .* S_noisy;
```

```
% Inverse STFT
```

```
filtered_song = istft(S_clean, Fs1, 'Window', window, 'OverlapLength', overlap, 'FFTLenght', nfft);
```

```
filtered_song = filtered_song / max(abs(filtered_song)); % Normalize
```

```
audiowrite('filtered_song_wiener_stft.wav', filtered_song, Fs1);
```

```
disp('✅ Wiener filtered song saved.');
```

```
% Extract Residual Noise
```

```
residual_noise = noisy_signal(1:length(filtered_song)) - filtered_song;
```



```
audiowrite('extracted_noise_stft.wav', residual_noise, Fs1);  
disp('✅ Extracted residual noise saved.');
```

```
%% Step 4: FFT Comparison
```

```
N = length(filtered_song);  
f_axis = (0:N-1)*(Fs1/N);  
fft_filtered = fft(filtered_song, N);  
fft_residual = fft(residual_noise, N);
```

```
% Plot each graph in a separate figure
```

```
% Plot 1: Original Song
```

```
figure;  
plot((0:length(song)-1)/Fs1, song);  
title('Original Song');  
xlabel('Time (s)');  
ylabel('Amplitude');
```

```
% Plot 2: Original Noise
```

```
figure;  
plot((0:length(noise)-1)/Fs1, noise);  
title('Original Noise');  
xlabel('Time (s)');  
ylabel('Amplitude');
```

```
% Plot 3: Mixed Noisy Signal
```

```
figure;  
plot((0:length(noisy_signal)-1)/Fs1, noisy_signal);  
title('Mixed Noisy Signal');  
xlabel('Time (s)');
```

```
ylabel('Amplitude');
```

```
% Plot 4: STFT Wiener Filtered Song
```

```
figure;
```

```
plot((0:length(filtered_song)-1)/Fs1, filtered_song);
```

```
title('Filtered Song (Wiener)');
```

```
xlabel('Time (s)');
```

```
ylabel('Amplitude');
```

```
% Plot 5: Extracted Residual Noise
```

```
figure;
```

```
plot((0:length(residual_noise)-1)/Fs1, residual_noise);
```

```
title('Extracted Residual Noise');
```

```
xlabel('Time (s)');
```

```
ylabel('Amplitude');
```

```
% Plot 6: FFT of Filtered Song
```

```
figure;
```

```
plot(f_axis(1:N/2), abs(fft_filtered(1:N/2)));
```

```
title('FFT of Filtered Song');
```

```
xlabel('Frequency (Hz)');
```

```
ylabel('Magnitude');
```

```
% Plot 7: FFT of Residual Noise
```

```
figure;
```

```
plot(f_axis(1:N/2), abs(fft_residual(1:N/2)));
```

```
title('FFT of Residual Noise');
```

```
xlabel('Frequency (Hz)');
```

```
ylabel('Magnitude');
```

% Plot 8: Power Spectrum of Noisy Signal vs Noise

figure;

hold on;

plot(f_axis(1:N/2), P_noisy(1:N/2), 'b', 'LineWidth', 1.5);

plot(f_axis(1:N/2), P_noise(1:N/2), 'r', 'LineWidth', 1.5);

title('Power Spectrum of Noisy Signal and Noise');

xlabel('Frequency (Hz)');

ylabel('Power');

legend('Noisy Signal', 'Noise');

hold off;

disp('🎉 All processing and plotting done!');

OUTPUT:

